

Competitive Programming

Lab Assignment 01[Week 01]

Name : **M. Sai Pallavi**
Hall Ticket No : **2303A54063**
Batch : **48**
Day : **Wednesday**

Question 1: Assignment 3: Maximum Profit Streak (Divide and Conquer)

Problem Statement

A company tracks daily profit changes as an array A (values can be negative). A “profit streak” is any non-empty contiguous subarray. Find the maximum possible sum of a profit streak using a divide-and-conquer approach (split into left, right, and crossing subproblems). Input Format The first line contains integer T. For each test case: - First line: N - Second line: N integers A1.

Output Format For each test case, print one integer: maximum subarray sum. Constraints - $1 \leq T \leq 20$ - $1 \leq N \leq 200000$ (sum of N over all test cases ≤ 200000) - $-10^9 \leq A_i \leq 10^9$

Sample Input

```
1
9 -2 1 -3 4 -1 2 1 -5 4
```

Expected Output 6

Code:

```
def max_crossing_sum(arr, left, mid, right):
    left_sum = float('-inf')
    curr_sum = 0
    for i in range(mid, left - 1, -1):
        curr_sum += arr[i]
        left_sum = max(left_sum, curr_sum)
    right_sum = float('-inf')
    curr_sum = 0
    for i in range(mid + 1, right + 1):
        curr_sum += arr[i]
        right_sum = max(right_sum, curr_sum)
```

```

    return left_sum + right_sum
def max_subarray_sum(arr, left, right):
    if left == right:
        return arr[left]
    mid = (left + right) // 2
    return max(
        max_subarray_sum(arr, left, mid),
        max_subarray_sum(arr, mid + 1, right),
        max_crossing_sum(arr, left, mid, right)
    )
T = int(input().strip())
for _ in range(T):
    data = list(map(int, input().split()))
    n = data[0]
    arr = data[1:]
    print(max_subarray_sum(arr, 0, n - 1))

```

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: Welcome, 2303A54063, CP_wed1, Create New Project, My Projects, Classroom (now), Learn Programming, Programming Questions, Upgrade, and Logout. The main editor area displays a C++ program named main.py. The program defines two functions: max_crossing_sum and max_subarray_sum. The max_crossing_sum function calculates the maximum sum of subarrays crossing the midpoint. The max_subarray_sum function uses a divide-and-conquer approach to find the maximum subarray sum. The program reads an integer T, then for each test case, it reads an integer N and a list of integers. The output shows the program finished with exit code 0.

```
1 def max_crossing_sum(arr, left, mid, right):
2     left_sum = float('-inf')
3     curr_sum = 0
4     for i in range(mid, left - 1, -1):
5         curr_sum += arr[i]
6         left_sum = max(left_sum, curr_sum)
7
8     right_sum = float('-inf')
9     curr_sum = 0
10    for i in range(mid + 1, right + 1):
11        curr_sum += arr[i]
12        right_sum = max(right_sum, curr_sum)
13
14    return left_sum + right_sum
15
16
17 def max_subarray_sum(arr, left, right):
18     if left == right:
19         return arr[left]
20
21     mid = (left + right) // 2
22
23     return max(
24         max_subarray_sum(arr, left, mid),
25         max_subarray_sum(arr, mid + 1, right),
26         max_crossing_sum(arr, left, mid, right)
27     )
28
29 T = int(input().strip())
30
31 for _ in range(T):
32     n = int(input().strip())
33     arr = list(map(int, input().split()))
34
35     print(max_subarray_sum(arr, 0, n - 1))
36
37 ...Program finished with exit code 0
38 Press ENTER to exit console.
```

Question 2:

Problem Statement

You are given N jobs. Each job takes exactly 1 unit of time. Job i has a deadline D_i and a profit P_i . If a job is completed on or before its deadline, its profit is earned; otherwise, it cannot be counted. You can perform at most one job at a time. Your task is to choose and schedule jobs to maximize total profit. For each test case, output: (1) the number of jobs completed (2) the maximum total profit.

Input Format The first line contains an integer T , the number of test cases. For each test case: - The first line contains an integer N . - The next N lines each contain two integers D_i and P_i .

Output Format For each test case, print two integers: jobs_done total_profit

Constraints - $1 \leq T \leq 20$ - $1 \leq N \leq 200000$ (sum of N over all test cases ≤ 200000) - $1 \leq D_i \leq 100000$ - $1 \leq P_i \leq 10^9$

Sample Input

1

5

2 100

1 19

2 27

1 25

3 15

Expected Output 3 142

Code:

```
def job_sequencing(jobs):  
    jobs.sort(key=lambda x: x[1], reverse=True)  
    max_deadline = max(job[0] for job in jobs)  
    slots = [-1] * (max_deadline + 1)  
    jobs_done = 0  
    total_profit = 0  
    for deadline, profit in jobs:  
        for t in range(min(deadline, max_deadline), 0, -1):  
            if slots[t] == -1:  
                slots[t] = profit  
                jobs_done += 1  
                total_profit += profit  
                break  
    return jobs_done, total_profit  
  
T = int(input().strip())  
for _ in range(T):  
    N = int(input().strip())  
    data = []  
    while len(data) < 2 * N:  
        data.extend(map(int, input().split()))  
    jobs = []  
    for i in range(0, 2 * N, 2):  
        jobs.append((data[i], data[i + 1]))  
    result = job_sequencing(jobs)  
    print(result[0], result[1])
```

Files

GDB online Debugger | Compile

onlinegdb.com/#

OnlineGDB

online compiler and debugger for c/c++

Welcome, 2303A54063 ▲

Create New Project

My Projects

Classroom new

Learn Programming

Programming Questions

Upgrade

Logout ▼



All the connections, none of the limits. Unlimited Okta and social connections on our Free Plan.

ADS VIA BUYSSELLADS

About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy

main.py

```
1 def job_sequencing(jobs):
2     jobs.sort(key=lambda x: x[1], reverse=True)
3
4     max_deadline = max(job[0] for job in jobs)
5     slots = [-1] * (max_deadline + 1)
6
7     jobs_done = 0
8     total_profit = 0
9
10    for deadline, profit in jobs:
11        for t in range(min(deadline, max_deadline), 0, -1):
12            if slots[t] == -1:
13                slots[t] = profit
14                jobs_done += 1
15                total_profit += profit
16                break
17
18    return jobs_done, total_profit
19
20    T = int(input().strip())
21
22    for _ in range(T):
23        N = int(input().strip())
24
25        data = []
26        while len(data) < 2 * N:
27            data.extend(map(int, input().split()))
28
29        jobs = []
30        for i in range(0, 2 * N, 2):
31            jobs.append((data[i], data[i + 1]))
32
```

input

```
1 19
2 27
1 25
3 15
3 142

...Program finished with exit code 0
Press ENTER to exit console.
```